

Teknik Exploitasi Use After Free di Linux Kernel 7.0-rc7 dengan Teknik slab_sheaf Union State Confusion

by: Antonius

Country: Indonesia

<https://www.bluedragonsec.com> – <https://github.com/bluedragonsecurity>

1. Latar Belakang

Linux kernel 7.0-rc7 menggunakan arsitektur SLUB Sheaves yang menggantikan struktur lama `kmem_cache_cpu`. Dalam arsitektur baru ini, setiap sheaf direpresentasikan oleh `struct slab_sheaf` yang dideclare di **mm/slub.c baris 404 di linux kernel 7.0 rc 7**.

Dari hasil analisa source code linux kernel 7 rc 7, Salah satu karakteristik desain yang paling menarik dari `struct slab_sheaf` adalah penggunaan union pada bagian awal struct. Union ini memungkinkan satu blok memori yang sama digunakan untuk tiga tujuan berbeda tergantung pada konteks. Teknik exploitasi linux kernel use after free dengan **slab_sheaf Union State Confusion** berkaitan langsung dengan ambiguitas yang muncul dari penggunaan union ini.

2. Anatomi Union di slab_sheaf

2.1 Deklarasi Union (mm/slub.c:404)

Berikut adalah deklarasi lengkap bagian union dari `struct slab_sheaf`:

```
struct slab_sheaf {
    union {
        struct rcu_head rcu_head; /* dipakai saat RCU free */
        struct list_head barn_list; /* dipakai saat ada di barn */
        struct {
            unsigned int capacity;
            bool pfmemalloc;
        }; /* dipakai saat prefill */
    };
    struct kmem_cache *cache;
    unsigned int size;
    int node;
    void *objects[];
};
```

Union ini tidak memberi nama alias tambahan — ketiga varian langsung bisa diakses melalui nama field masing-masing (rcu_head, barn_list, capacity, pfmemalloc).

2.2 Layout Memori Aktual dari Union

Meskipun ketiga varian union dideklarasikan secara terpisah, dalam memori semuanya menempati rentang byte yang sama persis, dimulai dari offset +0x00 struct slab_sheaf.

Offset	Ukuran	Nama Field (via rcu_head)	Nama Field (via barn_list)
+0x00	8 byte	rcu_head.func (pointer ke callback function)	barn_list.next (pointer ke elemen berikutnya)
+0x08	8 byte	rcu_head.next (pointer linkage internal RCU)	barn_list.prev (pointer ke elemen sebelumnya)

Kolom pertama pada tabel di atas menunjukkan bahwa rcu_head.func dan barn_list.next berada di offset yang identik (+0x00). Demikian pula rcu_head.next dan barn_list.prev sama-sama di +0x08.

Ini adalah konsekuensi langsung dari cara kerja union di C: semua anggota union berbagi alamat awal yang sama. Tidak ada pemisahan memori di antara ketiganya.

2.3 Definisi Tipe Masing-Masing Field

Memahami tipe masing-masing field penting untuk mengerti makna overlap tersebut:

```
/* struct rcu_head (dari kernel RCU infrastructure) */
struct rcu_head {
    void (*func)(struct rcu_head *head); /* +0x00: callback fn */
    struct rcu_head *next;                /* +0x08: RCU linkage */
};

/* struct list_head (doubly-linked list kernel standard) */
struct list_head {
    struct list_head *next; /* +0x00: pointer ke berikutnya */
    struct list_head *prev; /* +0x08: pointer ke sebelumnya */
};
```

Perbedaan paling kritikal ada di offset +0x00: dalam konteks rcu_head, byte di posisi ini adalah pointer ke fungsi callback. Dalam konteks barn_list, byte yang sama adalah pointer ke struct list_head berikutnya. Satu blok memori 8 byte yang sama diinterpretasikan sebagai dua hal berbeda tergantung dari mana kode membacanya.

3. State Machine Sebuah slab_sheaf

Nah ! agar tidak bingung tentang union ini, kita perlu terlebih dahulu paham bagaimana siklus hidup (lifecycle) sebuah slab_sheaf berlangsung di kernel 7.0-rc7.

3.1 State-State yang Mungkin

Sebuah slab_sheaf dapat berada dalam salah satu state berikut pada waktu tertentu:

State	Deskripsi	Union Digunakan Sebagai	Bagian Kode
PERCPU_MAIN	Menjadi main sheaf di slub_percpu_sheaves suatu CPU	Tidak digunakan	alloc_from_pcs, free_to_pcs
PERCPU_SPARE	Menjadi spare sheaf di slub_percpu_sheaves suatu CPU	Tidak digunakan	__pcs_replace_*_main
PERCPU_RCU	Menjadi rcu_free sheaf, menampung batch objek untuk RCU	Tidak digunakan	__kfree_rcu_sheaf
IN_BARN	Berada dalam barn (pool NUMA), linked via barn_list	barn_list (list_head)	barn_put_*, barn_get_*
RCU_PENDING	Sudah di-call_rcu(), menunggu grace period RCU selesai	rcu_head (callback + linkage)	call_rcu, rcu_free_sheaf

3.2 Transisi State dan Penggunaan Union

Yang penting adalah transisi antara state IN_BARN dan state RCU_PENDING. Saat sheaf dipindahkan dari barn ke jalur RCU:

1. Sheaf diambil dari barn: list_del(&sheaf->barn_list) dipanggil. Setelah ini, barn_list.next dan barn_list.prev berisi nilai dari saat sheaf ada di doubly-linked list barn.
2. Sheaf dimasukkan ke pcs->rcu_free: union masih berisi nilai lama dari barn_list, belum di-clear.

3. Saat sheaf penuh: `call_rcu(&sheaf->rcu_head, rcu_free_sheaf)` dipanggil (`mm/slub.c:5949`). Pada titik ini, RCU infrastructure akan menulis nilai barunya sendiri ke `rcu_head.func` dan `rcu_head.next` — menimpa `barn_list.next/prev`.

Arah sebaliknya juga berlaku. Dalam `rcu_free_sheaf()` (`mm/slub.c:5789`), setelah RCU grace period selesai, jika barn masih memiliki slot, sheaf dikembalikan ke barn:

```
/* mm/slub.c:5829 – setelah RCU callback selesai */
if (data_race(barn->nr_full) < MAX_FULL_SHEAVES) {
    barn_put_full_sheaf(barn, sheaf);    /* masuk barn lagi! */
    return;
}
```

Ini berarti satu `slab_sheaf` dapat berulang kali melewati siklus: `barn` → `rcu_free_sheaf` → `call_rcu` → grace period → barn lagi. Setiap perpindahan melibatkan perubahan interpretasi union dari `barn_list` ke `rcu_head` dan sebaliknya.

4. State yang Gantung ! Inti dari Teknik `slab_sheaf` Union State Confusion !

Teknik ini berpusat pada satu pertanyaan: apakah ada momen di mana kernel membaca atau menulis union `slab_sheaf` dengan interpretasi yang tidak konsisten dengan state aktual sheaf?

4.1 Tidak Ada State Flag

Struct `slab_sheaf` tidak memiliki field eksplisit yang merekam state mana yang sedang aktif. Tidak ada enum state, tidak ada flag `BARN_OR_RCU`. Kode kernel menentukan interpretasi yang benar hanya dari konteks aliran program, misalnya, jika sheaf baru saja keluar dari barn, maka `barn_list` yang valid. Jika sheaf baru saja di-`call_rcu()`, maka `rcu_head` yang valid.

Ini adalah pendekatan yang umum di kernel (yang dibuat dari bahasa C) dan **tidak salah secara desain selama state machine dijalankan dengan benar**. Tapi ini menciptakan **dependensi pada validitas urutan operasi**.

4.2 Window Antara `call_rcu` dan Penggunaan `barn_list`

Perhatikan alur berikut dalam `__kfree_rcu_sheaf()` (`mm/slub.c:5862`):

```
/* Langkah 1: sheaf diambil dari spare atau barn */
/* Pada titik ini union berisi nilai yang valid sebagai barn_list */
/* atau nilai random jika diambil dari spare */
```

```

/* Langkah 2: objek dikumpulkan ke objects[] */
rcu_sheaf->objects[rcu_sheaf->size++] = obj;

/* Langkah 3: saat sheaf penuh, call_rcu dipanggil */
if (rcu_sheaf->size >= s->sheaf_capacity) {
    pcs->rcu_free = NULL;
    rcu_sheaf->node = numa_mem_id();
    /* call_rcu akan overwrite rcu_head.func dan rcu_head.next */
    call_rcu(&rcu_sheaf->rcu_head, rcu_free_sheaf); /* slub.c:5949 */
}

```

Antara Langkah 1 dan Langkah 3, ada jendela di mana union berisi nilai lama yang mungkin berasal dari konteks barn. Jika pada jendela ini ada kode lain yang membaca union dengan interpretasi rcu_head (misalnya kode debug atau tracing), nilai yang dibaca adalah barn_list.next/prev bukan nilai rcu_head yang valid.

4.3 Skenario Re-entry ke Barn Setelah RCU

Skenario yang lebih menarik secara teoritis terjadi dalam rcu_free_sheaf() (mm/slub.c:5789 – linux kernel 7.0 rc 7). Setelah RCU grace period selesai dan callback dipanggil:

```

static void rcu_free_sheaf(struct rcu_head *head)
{
    sheaf = container_of(head, struct slab_sheaf, rcu_head);
    /* union sekarang berisi nilai rcu_head (dari RCU infrastructure) */

    if (__rcu_free_sheaf_prepare(s, sheaf))
        goto flush;

    /* KONDISI: jika barn masih ada slot */
    if (data_race(barn->nr_full) < MAX_FULL_SHEAVES) {
        barn_put_full_sheaf(barn, sheaf);
        /* barn_put_full_sheaf akan list_add(&sheaf->barn_list, ...) */
        /* list_add MENULIS KE barn_list.next dan barn_list.prev */
        /* yang berada di OFFSET YANG SAMA dengan rcu_head.func/next */
        return;
    }
}

```

Saat barn_put_full_sheaf() memanggil list_add(&sheaf->barn_list, ...), operasi ini menulis ke barn_list.next (offset +0x00) dan barn_list.prev (offset +0x08). Nilai yang ditulis adalah pointer ke list_head struct di dalam barn. Karena barn_list dan rcu_head berbagi offset yang sama, efek visualnya adalah nilai rcu_head.func (offset +0x00) tertimpa oleh pointer barn_list.

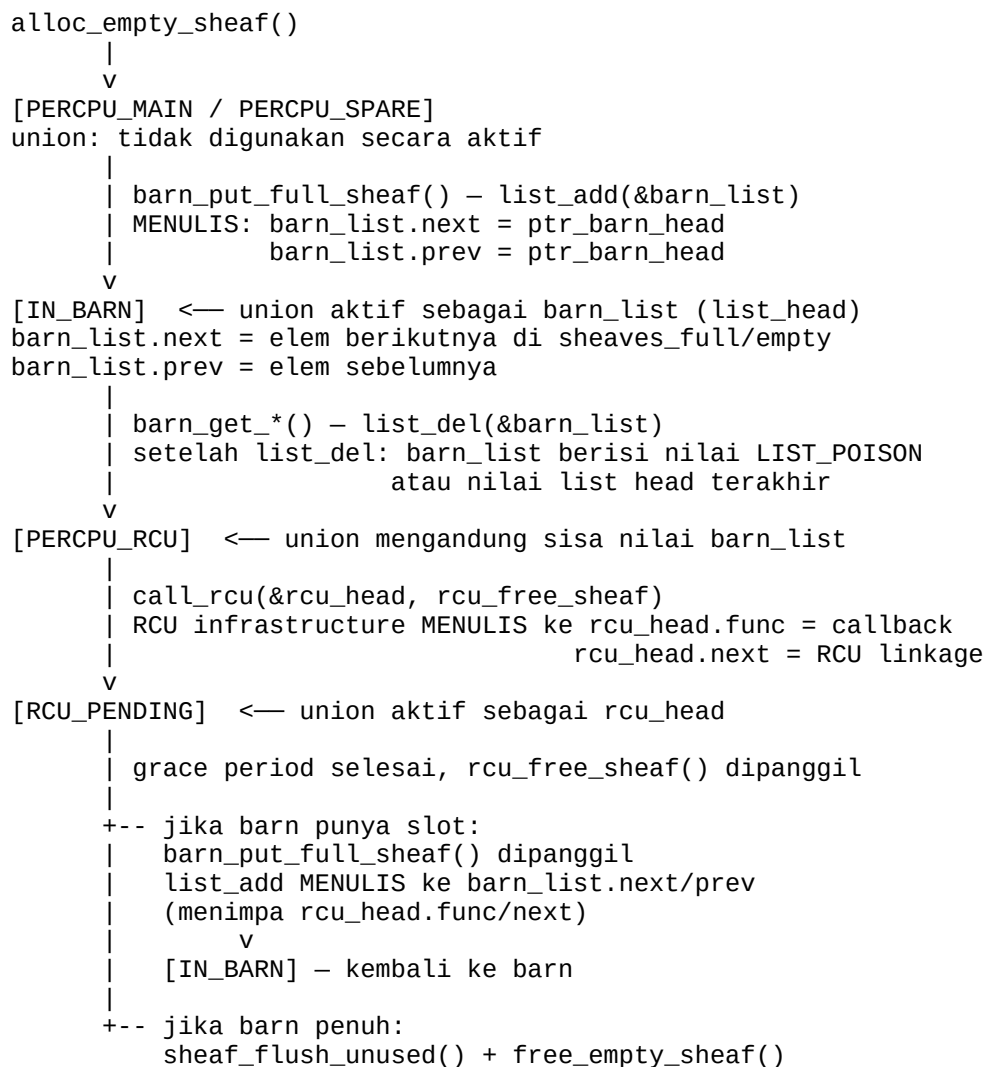
4.4 Jadi Apa Maknanya ?

Secara teori, ambiguitas ini menjadi perhatian dalam konteks berikut:

- Jika ada mekanisme yang memungkinkan pembacaan slab_sheaf pada saat yang salah, misalnya melalui kerentanan information leak atau race condition yang memberikan jendela observasi ke isi sheaf, pembaca tersebut mungkin menginterpretasikan nilai union secara berbeda dari yang dimaksud kernel.
- Jika ada primitif tulis yang dapat memodifikasi union slab_sheaf saat sheaf dalam state RCU_PENDING, nilai yang ditulis ke offset +0x00 akan diinterpretasikan oleh RCU infrastructure sebagai pointer ke callback function.
- Secara simetris: modifikasi di state IN_BARN ke offset +0x00 akan diinterpretasikan oleh kode barn sebagai barn_list.next, yang merupakan pointer ke elemen barn berikutnya dalam doubly-linked list.

5. Siklus Hidup Lengkap dan Titik Transisi

Untuk memahami teknik eksploitasi ini secara menyeluruh, berikut adalah diagram siklus hidup slab_sheaf dengan titik-titik transisi union yang relevan:



sheaf di-free ke kmalloc

6. Kondisi yang Diperlukan

Penting untuk menegaskan bahwa teknik ini bukan kerentanan yang dapat langsung dieksploitasi dari scratch. Ia adalah properti desain yang memperbesar dampak jika sudah ada bug lain.

Kondisi yang diperlukan :

- Primitif akses awal: harus ada kerentanan lain — seperti UAF (Use-After-Free), buffer overflow, atau race condition — yang memberikan kemampuan membaca atau menulis ke memori yang overlap dengan sebuah slab_sheaf.
- Timing yang tepat: akses tersebut harus terjadi saat sheaf berada dalam transisi antara state IN_BARN dan RCU_PENDING, atau saat rcu_free_sheaf() sedang mengembalikan sheaf ke barn.
- Pengetahuan lokasi sheaf: penyerang harus mengetahui alamat slab_sheaf target di memori. Ini umumnya membutuhkan information leak tersendiri.

Dalam istilah lain: teknik ini adalah amplifier, bukan initial vector. Ia mengubah primitif akses memori terbatas menjadi sesuatu yang berpotensi lebih signifikan karena nilai yang dimanipulasi punya makna khusus bagi infrastruktur kernel (RCU machinery atau barn list management).

7. Mitigasi & Pertimbangan Defensif

7.1 Pendekatan yang Ada

Kernel 7.0-rc7 memiliki beberapa lapisan perlindungan yang membatasi dampak praktis dari ambiguitas ini:

- KASAN (Kernel Address Sanitizer): pada build debug, KASAN mendeteksi akses ke memori yang belum diinisialisasi atau sudah di-free. Ini membantu mendeteksi kondisi transisi yang salah saat pengujian.
- lockdep: framework deteksi penggunaan lock yang salah. Beberapa transisi state melibatkan lock yang dilindungi lockdep, sehingga akses di luar konteks lock yang benar akan terdeteksi.
- Locking discipline: setiap barn operation dilindungi spin_lock_irqsave, dan setiap percpu operation dilindungi local_trylock. Ini membatasi window di mana state transisi dapat dieksploitasi.

7.2 Yang Tidak Ditangani

Mitigasi yang ada tidak secara eksplisit menandai state slab_sheaf atau memvalidasi interpretasi union. Pendekatan yang lebih kuat secara teoritis adalah:

- Menambahkan state field eksplisit ke struct slab_sheaf (enum atau bitfield) yang diupdate atomis saat transisi. Ini memungkinkan validasi runtime bahwa union diakses dengan interpretasi yang benar.
- Memisahkan union menjadi field terpisah dengan masa hidup (lifetime) yang jelas dan tidak overlap. Meski ini meningkatkan ukuran struct, ia menghilangkan ambiguitas sepenuhnya.

8. Kesimpulan

Teknik slab_sheaf Union State Confusion berpusat pada satu fakta arsitektur dari kernel 7.0-rc7: struct slab_sheaf menggunakan union untuk menumpuk rcu_head dan barn_list di offset memori yang sama, tanpa penanda state eksplisit yang mencegah akses dengan interpretasi yang salah.

Secara teoritis, ini menciptakan kondisi di mana nilai yang bermakna dalam satu konteks (pointer callback RCU atau pointer doubly-linked list barn) dapat dibaca atau ditulis melalui konteks yang berbeda jika ada bug lain yang memungkinkan akses ke isi slab_sheaf.